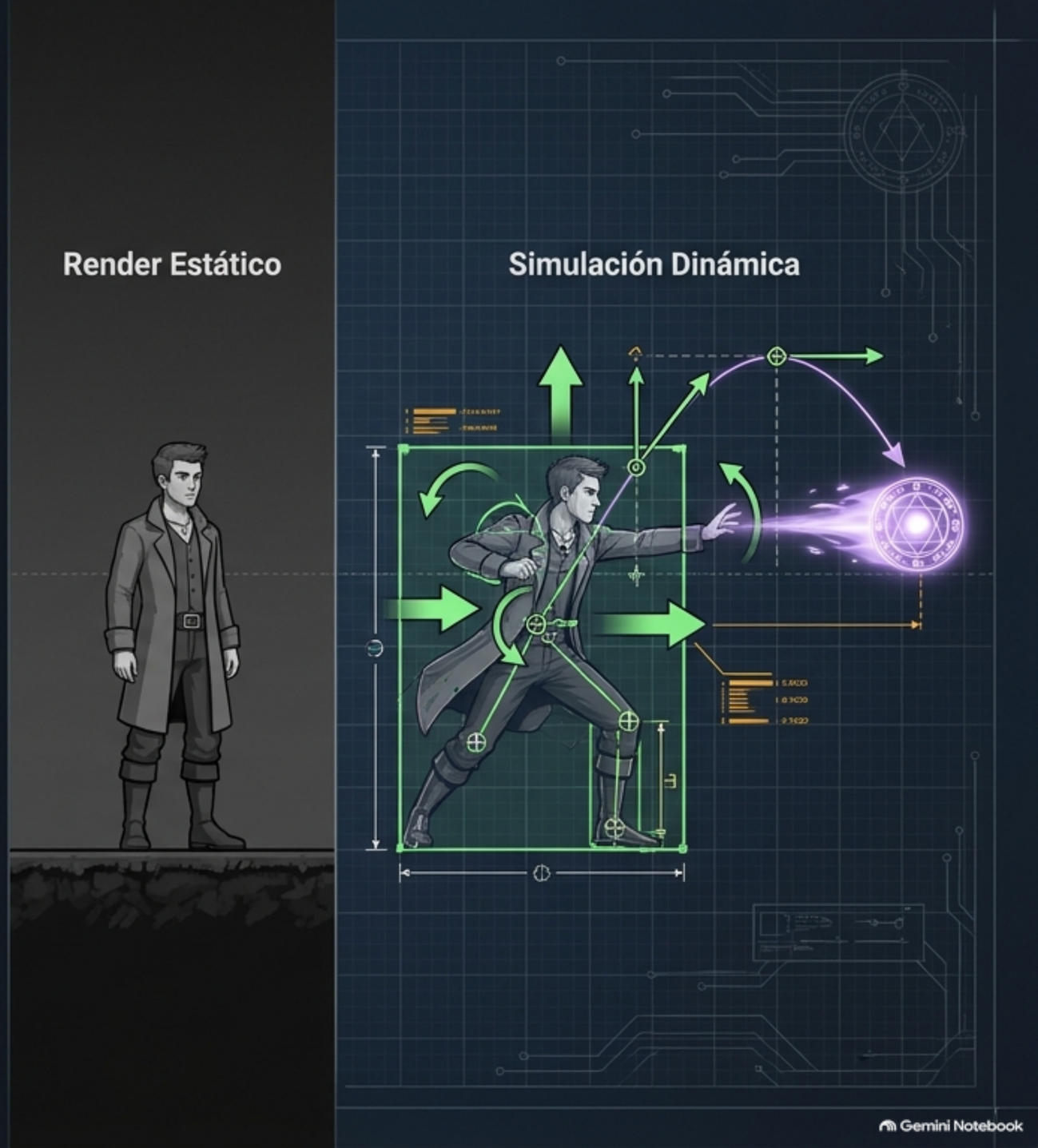
Lección 07: Físicas 2D, Proyectiles y Combate Balístico

- **De lo estático a lo dinámico:** Transformamos sprites inertes en entidades que caen, rebotan y se destruyen.
- **Estándares de la industria:** Implementación de técnicas usadas por Unity, Godot y Unreal (Bitmasks, AABB, Euler).
- **Objetivo final:** Construir un sistema balístico completo con trayectorias parabólicas y gestión de daño.

```
// Mapa Arquitectónico de la Lección
app/shared/src/commonMain/.../
├── Tags.kt        // Bitmasks
├── Geometry.kt    // AABB & Ángulos
├── Game.kt        // Físicas Euler
└── Damage.kt      // Sistema de Daño
```

Concepto Clave

Hasta hoy, el personaje era un dibujo en un lienzo vacío. Hoy construiremos desde cero las leyes de la física y el combate para responder: ¿qué pasa si salta o lanza un hechizo?



Gemini Notebook

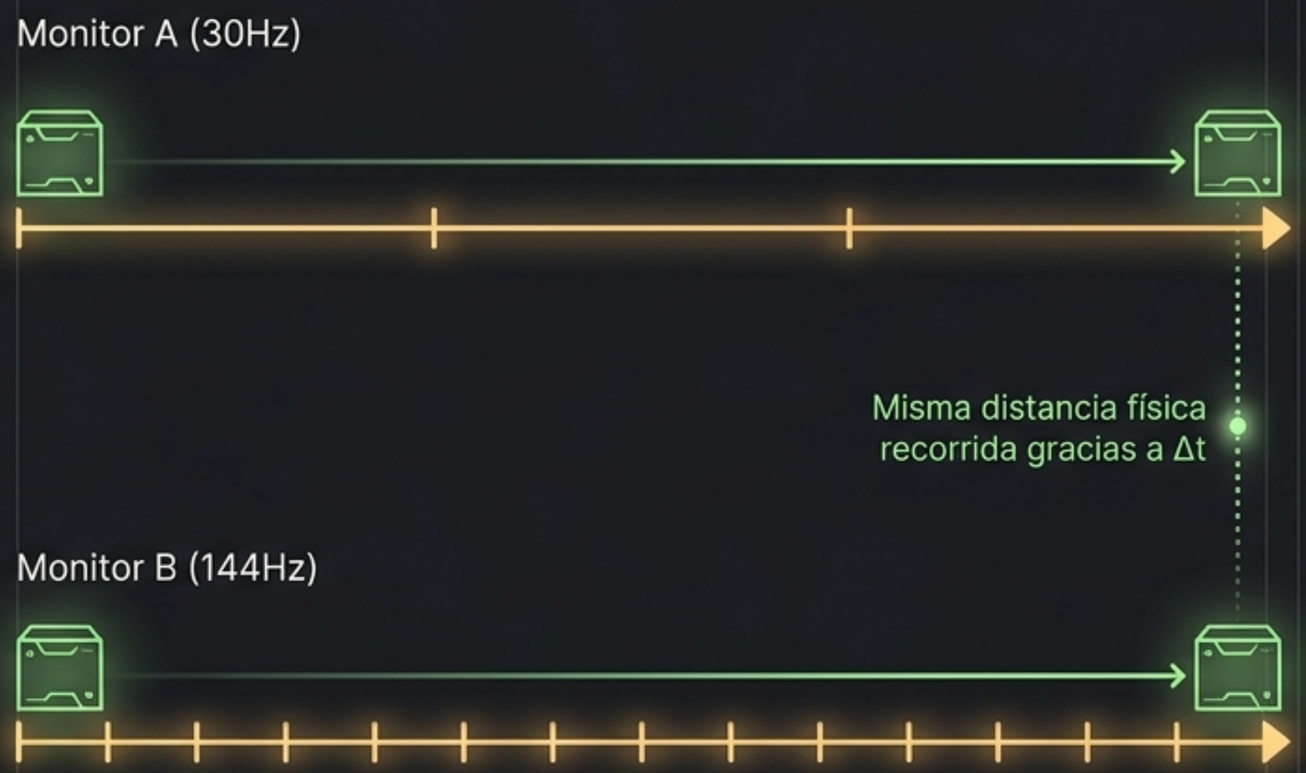
Delta Time: El Latido del Motor a 60 FPS

- **Independencia del Framerate**: Un juego debe correr a la misma velocidad en un monitor de 30Hz que en uno de 144Hz.
- **Conversión a Segundos**: Multiplicar siempre la velocidad por el tiempo transcurrido en segundos (Δt), no sumar directamente.
- **Fases del Frame**: Cada ≈ 16.6 ms el motor resuelve primero el movimiento/gravedad, y luego detecta las colisiones.

```
// Fórmula de independencia de framerate
val dtSeconds = dtMs / 1000.0
// La velocidad se escala por el tiempo
obj.x += obj.vx * dtSeconds
```

Regla de Oro

Nunca sumen velocidades directamente a la posición (x += vx). Multipliquen por el tiempo transcurrido para mantener la equidad física sin importar el hardware.



Gemini Notebook

Pilar 1: Filtrado de Colisiones Ultra-Rápido

- **El Problema:** Comparar Strings o colecciones crea objetos temporales que saturan el Garbage Collector provocando tirones (stutter).
- **La Solución:** Usar el operador binario de desplazamiento (1 shl N) para asignar categorías a bits individuales.
- **Magia Binaria:** Una sola operación lógica AND de la CPU resuelve si dos entidades chocan en nanosegundos.

Comparison Matrix

Strings / Enums	Bitmasks
	
Lento, genera basura en memoria	Operación de CPU en nanosegundos

```
// Categorías usando 1 shifted left N
const val CORPSE = 1 shl 0  // 000001
const val PLAYER = 1 shl 4  // 010000

fun canCollideWith(target: GameObject) =
    this !== target && (collisionMask and target.tags) != 0
```



Gemini Notebook

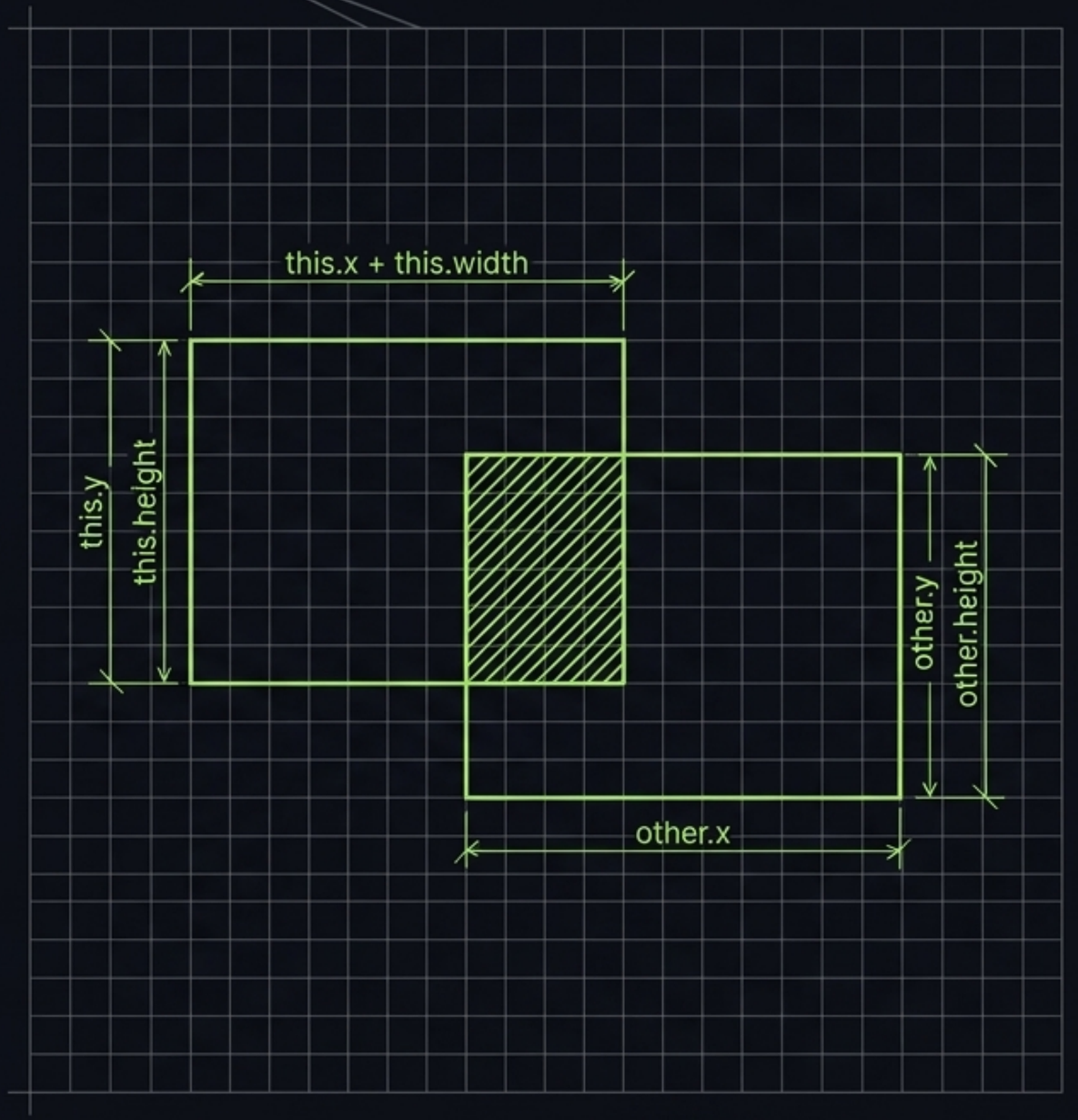
Pilar 2: Geometría 2D con Cajas Alineadas (AABB)

- **Primer filtro de espacio**: Mientras los Bitmasks filtran quién choca, el AABB determina cuándo chocan.
- **Simplicidad Matemática**: Al no tener rotación, verificar una colisión se reduce a 4 comparaciones numéricas simples.
- **Eficiencia**: Permite descartar el 99% de las colisiones lejanas en microsegundos antes de usar cálculos más pesados.

```
// Lógica de solapamiento AABB en ambos ejes
fun overlaps(other: Rect2D): Boolean {
    val overlapX = this.x < other.x + other.width && \
                   this.x + this.width > other.x
    val overlapY = this.y < other.y + other.height && \
                   this.y + this.height > other.y
    return overlapX && overlapY
}
```

Atención Geómetras

Usamos desigualdades estrictas (< y >). Si dos cajas comparten un borde exacto, se tocan pero no se penetran. Aún no es colisión física.



Gemini Notebook



Pilar 2 (Cont.): Del Ratón al Vector

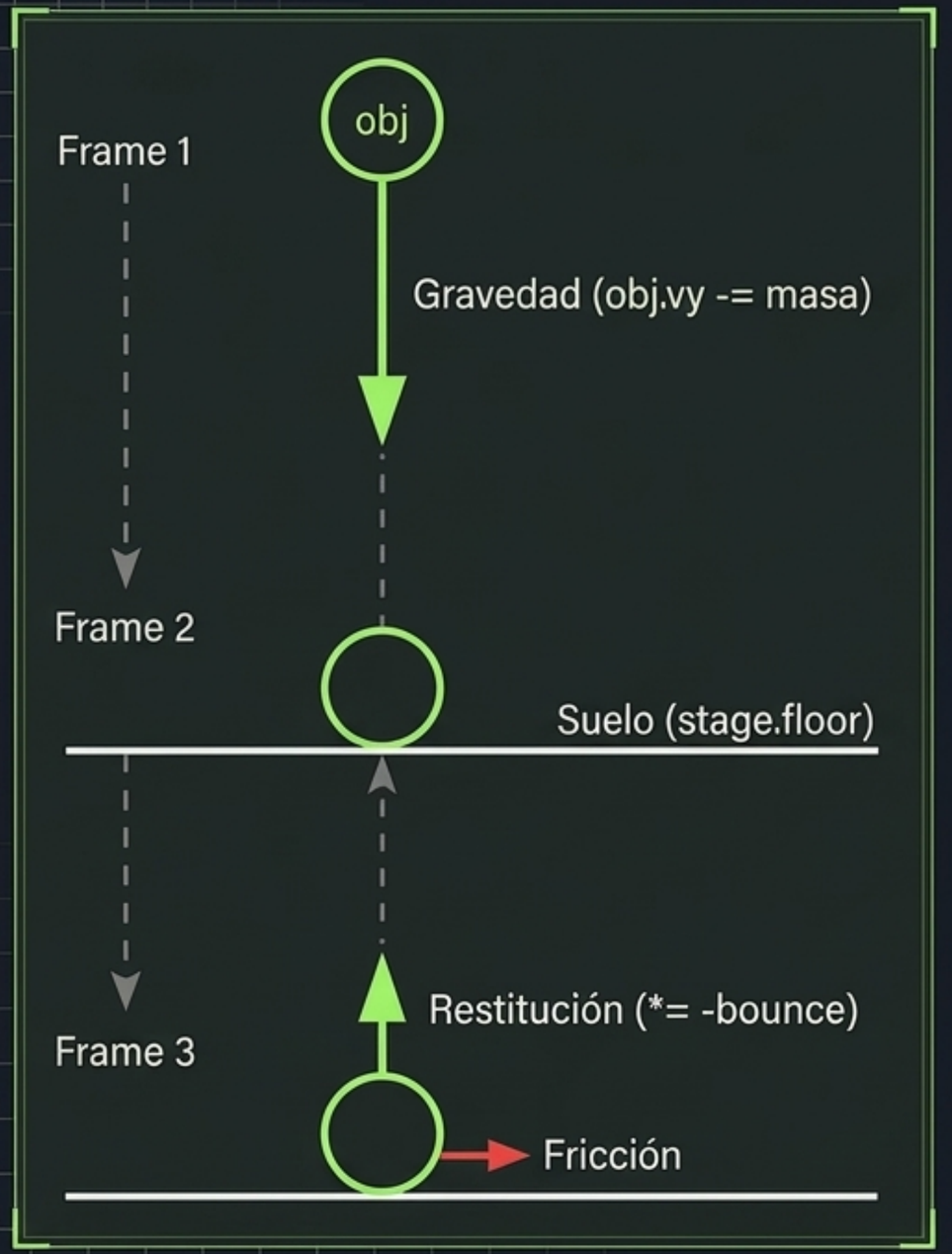
- **Ángulo Relativo:** Calculamos la dirección del disparo usando el arcotangente 2 (atan2) entre el cursor y el jugador.
- **Vectores Unitarios:** Convertimos ese ángulo puro en direcciones físicas (X, Y) usando Coseno y Seno.
- **Potencia Base:** El vector unitario resultante se escala multiplicándolo por la fuerza del proyectil.

```
// 1. Ángulo hacia el ratón
val theta = atan2(mouse.y - player.y, mouse.x - player.x)

// 2. Conversión a vector y escalado
val dirX = cos(theta) * basePower
val dirY = sin(theta) * basePower
```

¿Por qué atan2? Maneja divisiones por cero automáticamente y reconoce los 4 cuadrantes del plano cartesiano sin errores.

Gemini Notebook



Pilar 3: Simulando la Realidad Físicamente

- **Integración de Posición**: La velocidad modifica la posición en cada frame.
- **Gravedad como Aceleración**: Si el objeto tiene masa, se resta velocidad en el eje Y constantemente hacia abajo.
- **Límites y Restitución:** Al tocar el suelo, el objeto no lo atraviesa; se invierte su velocidad aplicándole un factor de rebote.

```
// Gravedad
if (obj.mass > 0 && obj.y > 0) obj.vy -= obj.mass * dt

// Rebote en el suelo
if (obj.y <= stage.floor) {
    obj.y = stage.floor
    obj.vy *= -obj.bounce // Invierte y atenúa velocidad
    obj.vx *= (1.0 - obj.friction) // Fricción horizontal
}
```

Nota de Diseño

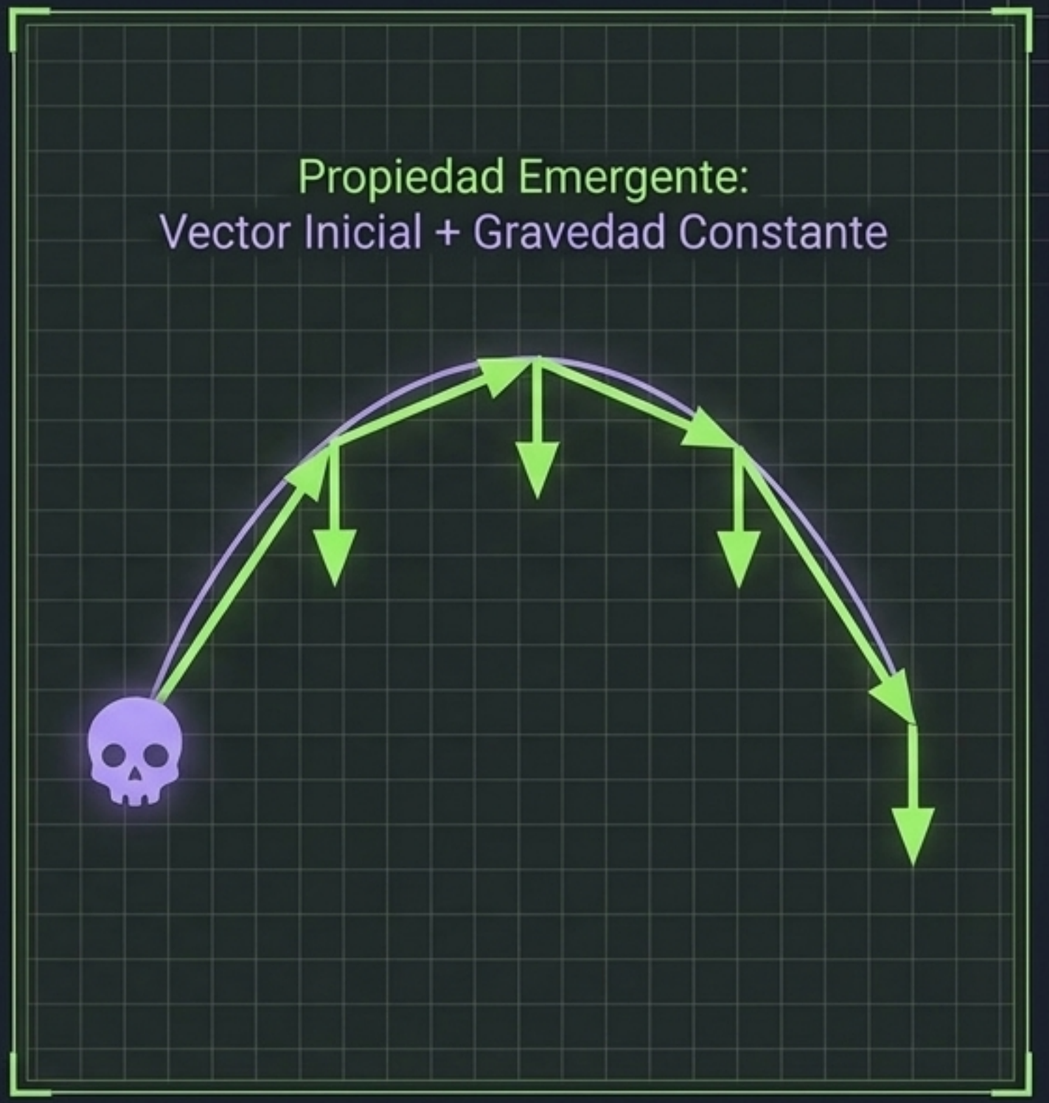
La Integración de Euler es muy intuitiva. Noten cómo la masa actúa literalmente como aceleración gravitacional. ¿Qué pasaría si el coeficiente de rebote fuera 1.0?

Gemini Notebook

Pilar 4: Balística Mágica y Parábolas Emergentes

- **Sin Fórmulas Complejas:** No programamos la curva parabólica; esta emerge naturalmente de combinar velocidad inicial y gravedad.
- **Gestión de Recursos (Cooldown):** Limitamos los disparos a través de un estado interno (ej. máximo 3 cargas, 1 seg de recarga).
- **Spawn (Generación):** El proyectil se crea en el centro del jugador y hereda la rotación matemática hacia el objetivo.

```
fun castSpell(): Boolean {
    if (spell.casts <= 0) return false
    spell.casts-- // Consumir carga
    val p = createSpell(dirX, dirY) // Usa trigonometría
    spawn(p, player.x + width/2, player.y + height/2)
    return true
}
```



Gemini Notebook

Síntesis: El Ciclo de Vida del Daño

Input del Jugador -> Spawn Proyectil

Bucle de Físicas y Parábola



Detección: Si canCollideWith y AABB overlaps son verdaderos, hay impacto.

Desacoplar es vital. Si mañana añadimos escudos mágicos, solo modificamos la función damage(), sin tocar los proyectiles.

Despawn

El proyectil se marca para destrucción, liberando memoria.



Desacoplamiento: gameSession.damage()

El proyectil no mata directamente; invoca el daño para que el motor central decida.

```
if (spell.overlaps(villager)) {
   spell.onCollision(villager)
   gameSession.damage(target = villager, amount = 1)
   if (spell.despawnOnCollision) despawn(spell)
}
```

HP > 0?

Sí

Recibe Daño / Sobrevive

No

Muerte del Aldeano

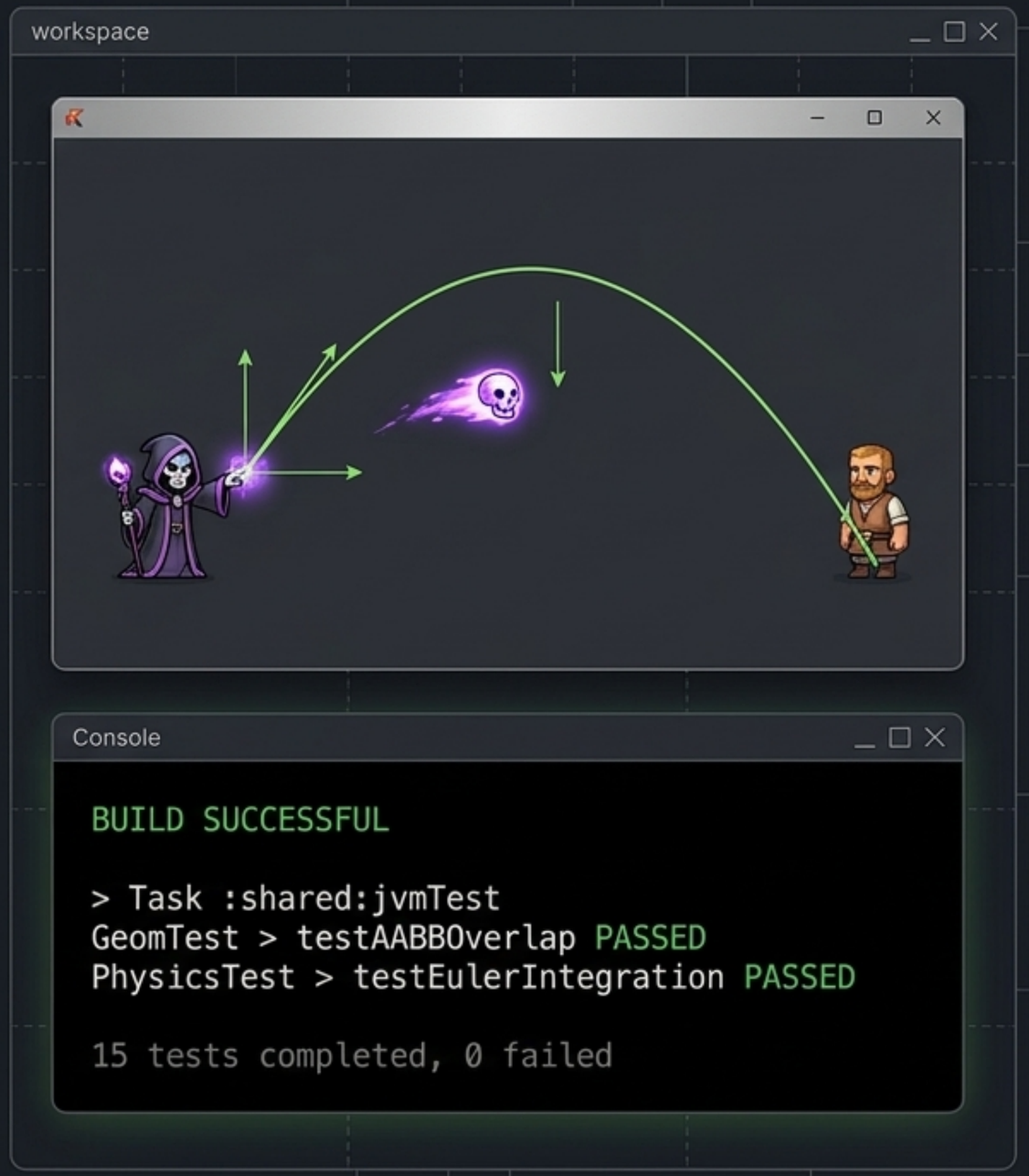
Gemini Notebook

Live Demo: Ejecutando el Motor KorGE

- **Validación Local:** Compilamos el proyecto en la JVM (Desktop) para ver la simulación a 60 FPS estables.
- **Interacción:** El usuario mueve el ratón para apuntar (Trigonometría) y hace clic para disparar (Balística y Euler).
- **Testing Automatizado:** Los motores serios requieren pruebas para asegurar que las matemáticas (AABB) no se rompan.

```
# Ejecutar la simulación visual interactiva
./gradlew :desktopApp:run

# Ejecutar tests matemáticos invisibles
./gradlew :shared:jvmTest
```



Gemini Notebook

Lo que Viene: El Arte de la Nigromancia

- **Hito Alcanzado:** Hemos construido desde cero los sistemas de físicas 2D que sustentan los juegos modernos de acción.
- **El Cambio de Estado:** Actualmente, al llegar a 0 HP el aldeano desaparece. En la lección 08, dejará un CORPSE.
- **La Mecánica Principal:** Usaremos las Bitmasks creadas hoy para que Norman resucite esos cadáveres como aliados UNDEAD.




```
// Próxima lección: Transición en lugar de eliminación
if (villager.hp <= 0) {
    villager.tags = Tags.CORPSE // De LIVING a CORPSE
    villager.sprite = corpseTexture
    // ¡Listo para ser revivido!
}
```

Memoria retenida. Listo para la resurrección.

Gemini Notebook